3-DOF leg kinematics: math and Python

Forward kinematics (FK) and inverse kinematics (IK) for one coxa–femur–tibia leg. Built on the derivation in your notebook (cosine rule, exterior angle, two height cases).

1. Conventions

Leg frame: origin on the coxa (hip) axis, **x** forward, **y** left, **z** up. All angles in radians in code.

• **C, F, T**: coxa, femur, tibia lengths.
• θ_1: coxa angle, rotation about the vertical axis (top view).
• θ_2: femur angle measured *above* the horizontal.
• θ_3: knee bend = angle the tibia turns *downward* relative to the extension line of the femur (your exterior angle). So the tibia's absolute angle is $\theta_2 - \theta_3$.

> Key idea: the femur and tibia always move in one vertical plane, and that plane **rotates with the coxa**. So a 3D problem splits into one top-view angle plus a 2D two-link problem.

2. Forward kinematics (angles → foot position)

Work in the leg plane first. Let r be the horizontal distance from the hip axis along the leg direction and z the height:

$$r = C + F\cos\theta_2 + T\cos(\theta_2 - \theta_3)$$

$$z = F\sin\theta_2 + T\sin(\theta_2 - \theta_3)$$

Then rotate the plane by the coxa angle to get the 3D point:

$$x = r\cos\theta_1, \quad y = r\sin\theta_1, \quad z = z$$

FK is just adding up link vectors. It has exactly one answer for each set of angles. That is why it is the easiest way to test your IK.

3. Inverse kinematics (foot position → angles)

Given the target (x, y, z):

Step 1: coxa angle. From above, the whole leg lies on one straight line toward the target.

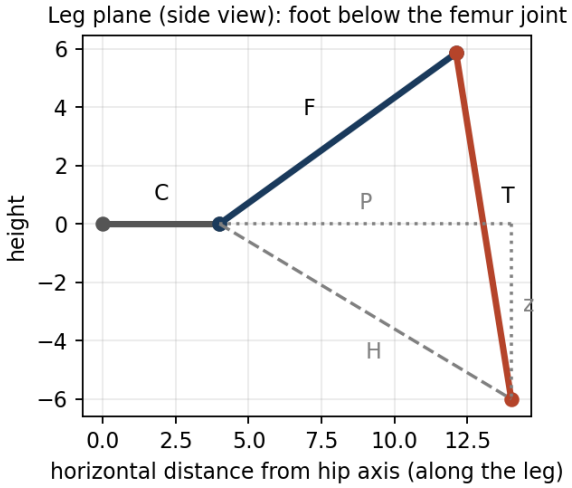
$$\theta_1 = \text{atan2}(y, x)$$

> This replaces the $x_a = C\cos\theta_1$ relation from the notebook, which used θ_1 to find θ_1.

Step 2: flatten into the leg plane. Measure the reach from the femur joint, not the hip:

$$P = \sqrt{x^2 + y^2} - C, \quad H = \sqrt{P^2 + z^2}$$

P is your "projection" but along the leg direction, not along the x axis (the two agree only when $\theta_1 = 0$). H is the straight line from femur joint to foot. z is the height of the foot relative to the femur joint (negative if the foot is below). If the femur joint is not at the hip height, subtract that offset from z.



Step 3: reachability. F, T and H form a triangle, so it only exists if

$$|F - T| \le H \le F + T$$

Step 4: cosine rule at the femur joint. The angle φ_1 between the femur and the line H comes from $T^2 = H^2 + F^2 - 2FH \cos \varphi_1$:

$$\varphi_1 = \cos^{-1}[\ (H^2 + F^2 - T^2) / (2FH)\ ]$$

The line H itself is tilted by φ_3 from the horizontal. Use atan2 so the sign is automatic (negative when the foot is below the joint):

$$\varphi_3 = \text{atan2}(z, P)$$

Step 5: femur and knee.

$$\theta_2 = \varphi_1 + \varphi_3$$

This single formula covers both of your notebook cases. Foot below: φ_3 is negative, so it equals your $\theta_2 = \varphi_1 - |\varphi_3|$. Foot above: it equals $\varphi_1 + \varphi_3$. No if-statement.

For the knee, apply the cosine rule to the interior angle at the knee: $H^2 = F^2 + T^2 - 2FT \cos(180° - \theta_3)$, which gives

$$\theta_3 = \cos^{-1}[\ (H^2 - F^2 - T^2) / (2FT)\ ]$$

This is the same as $\theta_3 = \varphi_1 + \varphi_2$ from your notes (exterior angle = sum of the two opposite interior angles). You do not need φ_2 at all.

4. Edge cases to remember

• **Unreachable target** (H outside [|F−T|, F+T]): raise an error. Never feed it to acos.
• **acos domain:** floating point can give 1.0000000002. Clamp to [−1, 1] before acos.
• **Elbow up vs down:** the other solution has the knee below the line H: $\theta_2 = \varphi_3 - \varphi_1$, θ_3 negative. This code always picks knee-up.
• **Servo mapping:** real servos have their own zero and direction. Convert with servo_angle = offset + sign × θ, set per joint and per leg (left and right legs usually mirror).
• **Target on the coxa axis** (x = y = 0): θ_1 is undefined.

5. Hand check (do this on paper first)

Take C = 0, F = T = 10 and target P = 10, z = 0. Then H = 10.

$$\varphi_1 = \cos^{-1}(100/200) = 60°, \quad \varphi_3 = 0° \ \Rightarrow\ \theta_2 = 60°$$

$$\theta_3 = \cos^{-1}((100 - 200)/200) = \cos^{-1}(-0.5) = 120°$$

FK check: $x = 10\cos 60° + 10\cos(60° - 120°) = 5 + 5 = 10$, and $z = 10\sin 60° + 10\sin(-60°) = 0$. ✓

6. Python code (leg_kinematics.py)

```
"""
leg_kinematics.py  -  FK and IK for one 3-DOF leg (coxa, femur, tibia)

Leg frame:  origin at the coxa (hip) axis, x forward, y left, z UP.
  theta1 : coxa angle, rotation about the vertical z axis (top view)
  theta2 : femur angle above the horizontal (radial) direction
  theta3 : knee bend = angle the tibia turns DOWNWARD relative to the
           extension line of the femur. Knee is "up" (elbow-up).
Links:  C = coxa length, F = femur length, T = tibia length
"""
import math

C, F, T = 4.0, 10.0, 12.0          # example lengths (same unit for everything)


def fk(theta1, theta2, theta3, C=C, F=F, T=T):
    """Joint angles (rad) -> foot position (x, y, z)."""
    r = C + F * math.cos(theta2) + T * math.cos(theta2 - theta3)  # horizontal reach
    z = F * math.sin(theta2) + T * math.sin(theta2 - theta3)      # height
    return r * math.cos(theta1), r * math.sin(theta1), z


def ik(x, y, z, C=C, F=F, T=T):
    """Foot position (x, y, z) -> joint angles (theta1, theta2, theta3) in rad.
    Raises ValueError if the point cannot be reached."""
    r = math.hypot(x, y)
    if r < 1e-9:
        raise ValueError("target is on the coxa axis: theta1 undefined")

    theta1 = math.atan2(y, x)              # step 1: aim the leg (top view)
    P = r - C                              # step 2: reach measured from femur joint
    H = math.hypot(P, z)                   #         straight distance hip->foot in leg plane

    if H > F + T or H < abs(F - T):        # step 3: reachability
        raise ValueError(f"unreachable: H={H:.2f}, allowed [{abs(F-T):.2f}, {F+T:.2f}]")

    phi1 = math.acos(_clamp((H*H + F*F - T*T) / (2*F*H)))   # step 4: cosine rule
    phi3 = math.atan2(z, P)                                  #         signed slope of H
    theta2 = phi1 + phi3                                     # step 5: femur
    theta3 = math.acos(_clamp((H*H - F*F - T*T) / (2*F*T)))  #         knee (exterior angle)
    return theta1, theta2, theta3


def _clamp(v):
    """Guard acos against tiny floating point overshoot such as 1.0000000002."""
    return max(-1.0, min(1.0, v))


if __name__ == "__main__":
    import random
    random.seed(1)
    worst = 0.0
    for _ in range(10000):
        t1 = random.uniform(-1.2, 1.2)
        t2 = random.uniform(-0.6, 1.4)
        t3 = random.uniform(0.2, 2.6)
        p = fk(t1, t2, t3)
        q = fk(*ik(*p))
        worst = max(worst, max(abs(a - b) for a, b in zip(p, q)))
    print(f"FK(IK(p)) vs p, worst position error over 10000 random poses: {worst:.2e}")

    # hand check from the notebook: F=T=10, C=0, P=10, z=0 -> 60, 120 degrees
    t = ik(10, 0, 0, C=0, F=10, T=10)
    print("hand check (deg):", [round(math.degrees(a), 3) for a in t])

    try:
        ik(40, 0, 0)
    except ValueError as e:
        print("unreachable test:", e)
```

7. Test results

```
FK(IK(p)) vs p, worst position error over 10000 random poses: 4.53e-14
hand check (deg): [0.0, 60.0, 120.0]
```

```
unreachable test: unreachable: H=36.00, allowed [2.00, 22.00]
```

The round trip FK(IK(p)) returns p to machine precision for 10,000 random poses, the hand check gives 0°, 60°, 120°, and an out-of-range target is rejected cleanly.

Your next tests: plot a half-ellipse foot path (the swing trajectory), run IK at each point, and check the angles stay inside your servo limits. That is the bridge to gait.